



Département des Technologies de l'information et de la
communication (TIC)
Filière Informatique et systèmes de communication
Orientation Sécurité informatique

Rapport de recherche

Caméras virtuelles et redirection de flux vidéo

Étudiant

Dylan Oliveira Ramos

Enseignant responsable

Prof. Jean-Marc Bost

Année académique

2025-2026

Yverdon-les-Bains, le 23.03.2026

1 Introduction

Pour pouvoir tromper les sites de vérification d'identité, il faut trouver un moyen de rediriger la vidéo générée vers une caméra détectée comme réelle par ceux-ci. La solution la plus simple est d'utiliser une caméra virtuelle, qui est un périphérique logiciel simulant une caméra physique.

1.1 Comparaison des solutions

Le tableau ci-dessous compare les différentes solutions permettant de créer des caméras virtuelles.

Solution	OS	Open source	Avantages	Inconvénients
v4l2loopback	Linux	Oui	Natif au noyau Linux, faible latence	Linux uniquement
OBS Studio	Linux, Windows, macOS	Oui	Abstraction de l'OS	Nécessite des actions manuelles, haute latence
UnityCapture	Windows	Oui	Caméra Windows native	Windows uniquement
CoreMedia IO	macOS	Non	Caméra macOS native	macOS uniquement

2 v4l2loopback

v4l2loopback est un module du noyau Linux permettant de créer des périphériques vidéo virtuels. Avec FFmpeg, il est ensuite possible de rediriger un flux vidéo vers ces périphériques, qui seront détectés comme des caméras réelles par les applications.

Les commandes qui vont suivre ont été effectuées sur une machine **Ubuntu 24.04**.

2.1 Installation

```
1 sudo apt install v4l2loopback-dkms v4l2loopback-utils ffmpeg
```

2.2 Création d'une caméra virtuelle

La commande ci-dessous crée une caméra virtuelle appelée VirtualCam :

```
1 sudo modprobe v4l2loopback devices=1 video_nr=2 card_label="VirtualCam"
exclusive_caps=1
```

- `modprobe v4l2loopback` : charge le module noyau.
- `devices=1` : crée 1 périphérique virtuel.
- `video_nr=2` : spécifie le numéro du périphérique (erreur s'il existe déjà).
- `card_label="VirtualCam"` : spécifie le nom du périphérique.
- `exclusive_caps=1` : rend la caméra compatible avec les applications (simule une caméra réelle).

Il est ensuite possible de lister les caméras disponibles :

```
1 v4l2-ctl --list-devices
```

2.3 Envoi d'un flux vidéo vers la caméra virtuelle

La commande ci-dessous joue la vidéo `video.mp4` en boucle sur la caméra virtuelle :

```
1 ffmpeg -re -stream_loop -1 -i video.mp4 -f v4l2 -pix_fmt yuv420p /dev/video2
```

- `re` : temps réel (simule une vraie caméra sans envoyer toutes les images en une fois).
- `stream_loop -1` : boucle indéfiniment.
- `i video.mp4` : spécifie la vidéo à lancer.
- `f v4l2` : spécifie le format de sortie.
- `pix_fmt yuv420p` : spécifie le format de la vidéo (utilisé par les vraies caméras).
- `/dev/video2` : spécifie le périphérique de sortie.

2.3.1 Rechargement du module

Avant chaque diffusion de vidéo, il faut recharger le module `v4l2loopback` pour éviter les problèmes de conflits. Cela garantit que la caméra virtuelle est correctement réinitialisée et prête à recevoir le flux vidéo :

```
1 sudo modprobe -r v4l2loopback
2 sudo modprobe v4l2loopback devices=1 video_nr=2 card_label="VirtualCam"
   exclusive_caps=1
```

2.4 Script d'automatisation

Le script ci-dessous automatise le processus de rechargement du module et de diffusion de la vidéo sur la caméra virtuelle :

```
1 #!/bin/bash
2
3 # Check if video file argument is provided
4 if [ $# -eq 0 ]; then
5     echo "Error: Please provide a video file path"
6     echo "Usage: $0 <video_file>"
7     exit 1
8 fi
9
10 VIDEO_FILE="$1"
11
12 # Check if the video file exists
13 if [ ! -f "$VIDEO_FILE" ]; then
14     echo "Error: Video file '$VIDEO_FILE' not found"
15     exit 1
16 fi
17
18 echo "Removing v4l2loopback module..."
19 sudo modprobe -r v4l2loopback
20
21 echo "Loading v4l2loopback module with virtual camera..."
22 sudo modprobe v4l2loopback devices=1 video_nr=2 card_label="VirtualCam"
23     exclusive_caps=1
24
25 # Small delay to ensure the device is ready
```

```
25 sleep 1
26
27 echo "Starting video stream to virtual camera..."
28 echo "Press Ctrl+C to stop streaming"
29 ffmpeg -re -stream_loop -1 -i "$VIDEO_FILE" -f v4l2 -pix_fmt yuv420p /dev/video2
```

3 OBS Studio

Sous Windows, contrairement à Linux, il n'existe pas d'outils en ligne de commande permettant de créer des caméras virtuelles. Pour pouvoir créer une caméra virtuelle, il faut soit développer un driver customisé¹, soit utiliser un logiciel proposant cette fonctionnalité. **OBS Studio** par exemple, utilise la scène comme caméra virtuelle et permet de rediriger un flux vidéo vers celle-ci.

Les instructions qui vont suivre ont été effectuées sur une machine virtuelle **Windows 10 22H2**.

3.1 Installation

Télécharger et installer **OBS Studio** : <https://obsproject.com/>. Une fois le logiciel installé, la caméra virtuelle est automatiquement créée et est disponible sous le nom de **OBS Virtual Camera**.

3.2 Envoi d'un flux vidéo vers la caméra virtuelle

1. Lancer **OBS Studio**.
2. Dans la section **Sources**, cliquer sur le bouton **+** et sélectionner **Media Source**.
3. Insérer le nom de la source.
4. Cocher **Local File** et **Loop**, sélectionner la vidéo à lancer dans **Local File**, puis cliquer sur **OK**.
5. Dans la section **Controls**, cliquer sur **Start Virtual Camera**.

La vidéo est maintenant diffusée en boucle sur la caméra virtuelle et est accessible aux applications, mais attention, la caméra n'apparaît pas dans le gestionnaire de périphériques Windows.

En effet, cela est dû au fait que c'est une caméra logicielle qui utilise le framework **DirectShow**, elle est donc enregistrée dynamiquement à l'exécution et n'est pas détectée comme un périphérique physique par le système d'exploitation². Il est néanmoins possible de vérifier qu'elle existe vraiment en utilisant **FFmpeg** pour lister les périphériques vidéo disponibles :

```
1 ffmpeg.exe -list_devices true -f dshow -i dummy
```

3.3 Automatisation

Comme vu dans le point précédent, la création de la source vidéo nécessite des actions manuelles, mais une fois celle-ci créée, il est tout à fait possible d'automatiser le lancement des vidéos via la ligne de commande. Comme pour Linux, il est possible d'envoyer un flux vidéo vers la caméra virtuelle en utilisant **FFmpeg**, cependant, pour que cela fonctionne avec **OBS Studio**, le flux doit être envoyé via le protocole **UDP**.

3.3.1 Création de la source vidéo

1. Lancer **OBS Studio**.

¹<https://medium.com/@sbonnet.dev/how-to-build-a-virtual-camera-under-linux-and-windows-7af0e6433796#3914>

²<https://medium.com/deelvin-machine-learning/how-does-obs-virtual-camera-plugin-work-on-windows-e92ab8986c4e#0878>

2. Dans la section **Sources**, cliquer sur le bouton **+** et sélectionner **Media Source**.
3. Insérer le nom de la source.
4. Décocher **Local File**.
5. Dans **Input**, entrer : `udp://127.0.0.1:1234` et cliquer sur **OK**.

Cette source vidéo est maintenant prête à recevoir un flux vidéo via UDP sur le port 1234.

3.3.2 Envoi d'un flux vidéo vers la caméra virtuelle

La commande ci-dessous joue la vidéo `video.mp4` en boucle sur la caméra virtuelle de **OBS Studio** :

```
1 ffmpeg.exe -re -stream_loop -1 -i video.mp4 -c:v libx264 -preset ultrafast -tune zerolatency -f mpegts "udp://127.0.0.1:1234?pkt_size=1316"
```

- `re` : temps réel (simule une vraie caméra sans envoyer toutes les images en une fois).
- `stream_loop -1` : boucle indéfiniment.
- `i video.mp4` : spécifie la vidéo à lancer.
- `c:v libx264` : encode la vidéo en H.264 (format compatible avec OBS).
- `preset ultrafast` : utilise le preset d'encodage le plus rapide (réduit la qualité mais permet d'avoir une latence plus faible).
- `tune zerolatency` : optimise l'encodage pour réduire la latence.
- `f mpegts` : spécifie le format de sortie (MPEG-TS est un format de conteneur compatible avec le streaming en direct).
- `udp://127.0.0.1:1234?pkt_size=1316` : spécifie l'adresse et le port de destination du flux UDP, ainsi que la taille des paquets (1316 est une taille courante pour le streaming en direct).

4 pyvirtualcam

La librairie Python `pyvirtualcam` permet d'envoyer un flux vidéo vers une caméra virtuelle existante, que ce soit sur Linux, Windows ou macOS. Elle a le grand avantage de gérer automatiquement les différentes étapes nécessaires pour que le flux vidéo soit correctement redirigé vers la caméra virtuelle. Ainsi, il est possible de s'affranchir de l'utilisation de **FFmpeg** et de la configuration de **OBS Studio**, le tout en étant compatible avec tous les systèmes d'exploitation.

4.1 Exemple d'utilisation sur Windows

Pour que `pyvirtualcam` fonctionne sur Windows, il faut avoir une caméra virtuelle disponible. Dans cet exemple, la caméra virtuelle de **OBS Studio** est utilisée (voir Chapitre 3.1 pour l'installation).

4.1.1 Création de l'environnement virtuel

```
1 python -m venv .venv
2 .venv\Scripts\activate
3 pip install pyvirtualcam opencv-python
```

4.1.2 Exemple de code

Source : <https://github.com/letmaik/pyvirtualcam/blob/main/examples/video.py>

```
1 # This script plays back a video file on the virtual camera.
2 # It also shows how to:
3 # - select a specific camera device
```

```
4 # - use BGR as pixel format
5
6 import argparse
7 import pyvirtualcam
8 from pyvirtualcam import PixelFormat
9 import cv2
10
11 parser = argparse.ArgumentParser()
12 parser.add_argument("video_path", help="path to input video file")
13 parser.add_argument("--fps", action="store_true", help="output fps every
14 second")
15 parser.add_argument("--device", help="virtual camera device, e.g. /dev/video0
16 (optional)")
17 args = parser.parse_args()
18
19 video = cv2.VideoCapture(args.video_path)
20 if not video.isOpened():
21     raise ValueError("error opening video")
22 length = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
23 width = int(video.get(cv2.CAP_PROP_FRAME_WIDTH))
24 height = int(video.get(cv2.CAP_PROP_FRAME_HEIGHT))
25 fps = video.get(cv2.CAP_PROP_FPS)
26
27 with pyvirtualcam.Camera(width, height, fps, fmt=PixelFormat.BGR,
28                          device=args.device, print_fps=args.fps) as cam:
29     print(f'Virtual cam started: {cam.device} ({cam.width}x{cam.height} @
30 {cam.fps}fps)')
31     count = 0
32     while True:
33         # Restart video on last frame.
34         if count == length:
35             count = 0
36             video.set(cv2.CAP_PROP_POS_FRAMES, 0)
37
38         # Read video frame.
39         ret, frame = video.read()
40         if not ret:
41             raise RuntimeError('Error fetching frame')
42
43         # Send to virtual cam.
44         cam.send(frame)
45
46         # Wait until it's time for the next frame
47         cam.sleep_until_next_frame()
48
49         count += 1
```

4.1.3 Utilisation

```
1 python video.py video.mp4
```

La vidéo video.mp4 est maintenant diffusée en boucle sur la caméra virtuelle de OBS Studio.